

Practice Question Set

1. What is a function?
 2. What are the two categories of functions in C?
 3. What is the role of `main()` function?
 4. What are the three pillars of a function?
 5. What is a function declaration?
 6. What is a function definition?
 7. What is a function call?
 8. What is the purpose of a function prototype?
 9. What are actual arguments and formal arguments?
 10. What are the four approaches to function architecture?
 11. What is a function with no arguments and no return value?
 12. What is a function with arguments and no return value?
 13. What is a function with no arguments and a return value?
 14. What is a function with arguments and a return value?
 15. What is the purpose of the `void` keyword in functions?
-

Important Interview/Viva Questions

16. Difference between declaration, definition, and function call.
17. Difference between actual arguments and formal arguments.

18. Difference between library functions and user-defined functions.
 19. Difference between TNRN and TSRS functions.
 20. Why is `main ()` called a special function?
 21. Why is a semicolon used after function declaration but not after function definition?
 22. Why are functions used?
 23. Can a function return more than one value?
 24. What happens when a function is called?
 25. What is the difference between `void add ()` and `int add ()` ?
-

Answers :

1. What is a function?

Answer:

A function is a self-contained block of code designed to perform a specific task and can be reused whenever needed.

2. What are the two categories of functions in C?

Answer:

1. Library (Predefined) Functions
2. User-Defined Functions

Examples:

`printf()`, `scanf()`, `sqrt()`

and

`add()`, `factorial()`, `area()`

3. What is the role of `main()` function?

Answer:

`main()` is the starting point of every C program. Program execution begins from `main()`.

4. What are the three pillars of a function?

Answer:

1. Function Declaration
 2. Function Call
 3. Function Definition
-

5. What is a function declaration?

Answer:

A function declaration tells the compiler that a function exists.

Syntax:

```
return_type function_name(parameters);
```

Example:

```
int add();
```

6. What is a function definition?

Answer:

A function definition contains the actual code or body of the function.

Example:

```
int add()  
{  
    return 10+20;  
}
```

7. What is a function call?

Answer:

A function call executes the function.

Example:

```
add();
```

8. What is the purpose of a function prototype?

Answer:

A function prototype informs the compiler about:

- Function name
- Return type
- Parameters

before the function is used.

9. What are actual arguments and formal arguments?

Answer:

Actual Arguments

Values passed during function call.

Example:

```
add(x, y);
```

Here, x and y are actual arguments.

Formal Arguments

Variables receiving values inside the function.

```
int add(int a, int b)
```

Here, `a` and `b` are formal arguments.

10. What are the four approaches to function architecture?

Answer:

1. Takes Nothing Returns Nothing (TNRN)
 2. Takes Something Returns Nothing (TSRN)
 3. Takes Nothing Returns Something (TNRS)
 4. Takes Something Returns Something (TSRS)
-

11. What is TNRN?

Answer:

A function with no arguments and no return value.

Example:

```
void add()  
{  
    printf("Hello");  
}
```

12. What is TSRN?

Answer:

A function with arguments but no return value.

Example:

```
void add(int a, int b)  
{
```

```
    printf("%d",a+b);  
}
```

13. What is TNRS?

Answer:

A function with no arguments but returns a value.

Example:

```
int add()  
{  
    return 10+20;  
}
```

14. What is TSRS?

Answer:

A function with arguments and returns a value.

Example:

```
int add(int a,int b)  
{  
    return a+b;  
}
```

15. What is the purpose of void?

Answer:

void indicates that a function does not return any value.

Example:

```
void display()  
{  
}
```

Important Interview Answers

16. Difference between Declaration, Definition, and Function Call

Declaration	Definition	Function Call
Announces function	Contains body	Executes function
Ends with ;	No semicolon	Invokes function
Example: <code>int add(); int add() {} add();</code>		

17. Difference between Actual and Formal Arguments

Actual Arguments	Formal Arguments
Passed during call	Declared in definition
x, y	a, b
Exist in calling function	Exist inside called function

18. Difference between Library and User-defined Functions

Library Functions	User-defined Functions
Built-in functions	Programmer-created
<code>printf()</code> , <code>scanf()</code>	<code>add()</code> , <code>factorial()</code>
Available through header files	Written by user

19. Difference between TNRN and TSRS

TNRN	TSRS
No arguments	Arguments present
No return value	Returns value
Least flexible	Most flexible

20. Why is `main()` called a special function?

Answer:

Because every C program starts execution from `main()`, and it is automatically called by the operating system.

21. Why is a semicolon used after declaration but not after definition?

Answer:

Declaration is only a statement and must end with ; .

Example:

```
int add();
```

Definition contains a block enclosed in braces and therefore does not require a semicolon.

```
int add()  
{  
}
```

22. Why are functions used?

Answer:

Functions provide:

- Code reusability
 - Modularity
 - Reduced code duplication
 - Easy debugging
 - Better readability
-

23. Can a function return more than one value?

Answer:

No.

A function can directly return only one value using `return`.

Multiple values are usually returned using arrays, structures, or pointers.

24. What happens when a function is called?

Answer:

- Memory is allocated to the function.
- Statements inside the function execute.
- After completion, memory is released.

- Control returns to the calling location.

25. Difference between `void add()` and `int add()`

`void add()`

Returns nothing

No return statement required

Used for displaying output

`int add()`

Returns an integer

Requires return statement

Used for calculations